SOFTWARE PROJECT FINAL REPORT

PetPulse Web

Team Members
William Hynds
Renad Nofal

Course: CIS434 Software Engineering
Date: April 30, 2026

Table of Content

1. Introduction..... 5
1.1 Purpose and Scope..... 5
1.2 Product Overview..... 5
1.3 Structure of the Document..... 5
1.4 Terms, Acronyms, and Abbreviations..... 6
2. Project Management Plan..... 7
2.1 Project Organization..... 7
2.2 Lifecycle Model Used..... 7
2.3 Risk Analysis..... 7
2.4 Hardware and Software Resource Requirements..... 8
Hardware..... 8
Software..... 8
2.5 Deliverables and Schedule..... 8
3. Requirement Specifications..... 9
3.1 Stakeholders for the System..... 9
3.2 Use Cases..... 9
3.2.1 Graphic Use Case Model..... 9
3.2.2 Textual Description for Each Use Case..... 10
3.3 Rationale for the Use Case Model..... 11
3.4 Non-Functional Requirements..... 11
4. Architecture..... 12
4.1 Architectural Style(s) Used..... 12
4.2 Architectural Model (Components and Interactions)..... 12
4.3 Technology, Software, and Hardware Used..... 13
4.4 Rationale for Architectural Style and Model..... 14
5. Design..... 14
5.1 User Interface Design..... 14
5.1.1 Navigation Bar..... 14
5.1.2 Home Page..... 14
5.1.3 About Page..... 14
5.1.4 Features Page..... 15
5.1.5 Subscribe Page..... 15
5.1.6 User Settings Page..... 15
5.1.7 Responsive Design..... 15
5.2 Components Design (Static and Dynamic Models)..... 15
Static Component Structure..... 15
Dynamic Component Behavior..... 16

5.3 Database Design..... 16
auth.users (Supabase-Managed)..... 16
public.profiles (Application-Managed)..... 16
5.4 Rationale for Detailed Design Models..... 17
5.5 Traceability from Requirements to Detailed Design Models..... 17
6. Test Management..... 18
6.1 Complete List of System Test Cases..... 18
6.1.1 Pages and Navigation..... 18
6.1.2 Authentication..... 18
6.1.3 User Settings..... 18
6.1.4 External API and Breed Cards..... 19
6.1.5 UI Components and Theme..... 19
6.1.6 Error Handling and Build Checks..... 19
6.2 Traceability of Test Cases to Use Cases..... 19
6.3 Techniques Used for Test Case Generation..... 20
6.4 Test Results and Assessments..... 20
6.5 Defect Reports..... 21
7. Conclusion..... 21
7.1 Outcomes of the Project..... 21
7.2 Lessons Learned..... 21
7.3 Future Development..... 22

List of Tables

Table	Description	Section
Table 1	Terms, Acronyms, and Abbreviations	1.4
Table 2	Project Organization	2.1
Table 3	Risk Analysis	2.3
Table 4	Software Resource Requirements	2.4
Table 5	Deliverables and Schedule	2.5
Table 6	Stakeholders	3.1
Table 7	UC-01: Browse Product Information	3.2.2
Table 8	UC-02: Register for an Account	3.2.2
Table 9	UC-03: Login to Account	3.2.2
Table 10	UC-04: Complete Mock Subscription	3.2.2
Table 11	Non-Functional Requirements	3.4
Table 12	Architectural Components – Presentation Tier	4.2
Table 13	Technology, Software, and Hardware Used	4.3
Table 14	Responsive Design Viewports	5.1.7
Table 15	Dynamic Component Behavior	5.2
Table 16	Database Design – auth.users	5.3
Table 17	Database Design – public.profiles	5.3
Table 18	Requirements Traceability Matrix	5.5
Table 19	Test Cases – Pages and Navigation	6.1.1
Table 20	Test Cases – Authentication	6.1.2
Table 21	Test Cases – User Settings	6.1.3
Table 22	Test Cases – External API and Breed Cards	6.1.4
Table 23	Test Cases – UI Components and Theme	6.1.5
Table 24	Test Cases – Error Handling	6.1.6
Table 25	Traceability of Test Cases to Use Cases	6.2
Table 26	Test Results and Assessments	6.4
Table 27	Defect Reports	6.5

1. Introduction

1.1 Purpose and Scope

This report documents the design, development, and testing of PetPulse Web, a full-stack promotional and educational website. The PetPulse Web application serves as the primary digital platform for the PetPulse smart pet health monitoring system, a wearable harness designed to track pet health over time and provide live health statistics such as step count.

The purpose of this document is to consolidate all project phases, from requirements gathering through architecture, design, implementation, and testing into a single final reference. The scope of the system is limited to the website and its third-party service integrations; it does not include the physical PetPulse hardware device or the companion mobile application.

1.2 Product Overview

PetPulse Web is a modern, responsive web application built with React and Next.js. It uses Supabase for user authentication and database management, integrates with The Dog API and The Cat API for educational pet health content, and employs Stripe in test mode for a mock subscription workflow.

Key capabilities of the website include:

- A landing page that introduces the PetPulse system and its value proposition.
- An educational features page displaying dog and cat breed data fetched dynamically from external APIs.
- User account registration with email verification, login, session persistence, and profile management via Supabase.
- A subscribe page presenting Basic and Pro pricing plans with mock Stripe checkout integration.
- A fully responsive design supporting desktop, tablet, and mobile viewports with light and dark theme switching.

1.3 Structure of the Document

This report is organized into the following major sections:

- Section 1: Introduction: Purpose, scope, product overview, and terminology.
- Section 2: Project Management Plan, Team organization, lifecycle model, risk analysis, resource requirements, and schedule.
- Section 3: Requirement Specifications: Stakeholders, use cases, and non-functional requirements.

- Section 4: Architectural style, component interactions, and technology stack.
- Section 5: Design: UI design, component design, database design, and traceability.
- Section 6: Test Management, Test cases, traceability, techniques, results, and defect reports.
- Section 7: Conclusions, Outcomes, lessons learned, and future development.

1.4 Terms, Acronyms, and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface
CSR	Client-Side Rendering
CTA	Call-To-Action
JWT	JSON Web Token (used for secure session management)
OTP	One-Time Password
Next.js	React-based full-stack web framework supporting SSR and CSR
RLS	Row Level Security – Supabase's per-row access control
SRS	Software Requirements Specification
SSR	Server-Side Rendering
Supabase	Open-source Firebase alternative providing PostgreSQL, Auth, and REST APIs
Stripe	Payment processing platform; used in test/mock mode for this project
The Dog API	Third-party RESTful API providing dog breed data
The Cat API	Third-party RESTful API providing cat breed data
UC	Use Case
UI	User Interface

2. Project Management Plan

2.1 Project Organization

Team Member	Primary Responsibilities
William Hynds	Full-stack architecture, Supabase integration, authentication flows (sign-up, OTP, login, JWT), Next.js routing, SSR configuration, API integrations (The Dog API, The Cat API), test specification authoring.
Renad Nofal	Frontend component development (React/shadcn), UI design and Tailwind CSS styling, responsive layout, Stripe mock integration.

2.2 Lifecycle Model Used

The project followed an iterative and incremental development lifecycle, organized around feature areas rather than strict waterfall phases. This approach was chosen to accommodate the short development timeline (~7 weeks) and the need to integrate multiple external services (Supabase, The Dog API, The Cat API, Stripe) progressively.

development phases:

1. Project setup, tooling, and requirements gathering (Week 1)
2. Architecture design and Supabase/database configuration (Week 2)
3. Core page development and SSR implementation (Week 3)
4. Authentication flows and client-side components (Week 4)
5. API integrations, subscribe page, and UI polish (Week 5)
6. Responsive design, theme switching, and bug fixes (Week 6)
7. Testing, defect resolution, and documentation (Week 7)

2.3 Risk Analysis

Risk	Likelihood	Impact	Mitigation
Third-party API unavailability (The Dog API, The Cat API)	Medium	Medium	Implement loading states, error fallbacks, and graceful degradation messages on all API-dependent pages.
Stripe test mode not reflecting real payment behavior	Low	Low	Clearly label all checkout flows as mock/test; no real credentials collected.

2.4 Hardware and Software Resource Requirements

Hardware

- No dedicated server hardware required; the application is deployed via a hosting provider.

Software

Tool / Service	Purpose
Next.js (App Router)	Full-stack React framework for SSR, routing, and bundling
React	Frontend UI library
shadcn/ui + Tailwind CSS	Component library and utility-first CSS framework
Supabase	Authentication, PostgreSQL database, and REST API (free plan)
The Dog API	RESTful API for dog breed and health data
The Cat API	RESTful API for cat breed data
Stripe (Test Mode)	Mock subscription checkout workflow
npm	Package manager
Git / GitHub	Version control and collaboration
Google Chrome, Firefox, Safari	Primary testing browsers

2.5 Deliverables and Schedule

Deliverable	Target Week	Status
Software Requirements Specification (SRS)	Week 1–2	Complete
Architecture and Design Specification	Week 2–3	Complete
Core page implementations (Home, About, Features, Subscribe)	Week 3–4	Complete
Authentication system (Signup/OTP/Login/Logout)	Week 4	Complete
User settings page (name, password, email update)	Week 5	Complete
External API integration (Dog API, Cat API)	Week 5	Complete
Stripe mock checkout integration	Week 5–6	Complete (mock/placeholder)
Responsive design and theme switching	Week 6	Complete
Test Specification document	Week 7	Complete

Deliverable	Target Week	Status
Software Project Final Report	Week 7	Complete

3. Requirement Specifications

3.1 Stakeholders for the System

Stakeholder	Role / Interest
Pet Owners (General Public)	Primary end users. Seek accessible information about the PetPulse harness, may register accounts and explore subscription options.
Development Team (William Hynds, Renad Nofal)	Responsible for design, implementation, testing, and deployment of the website.
Supabase	External service provider for authentication and database. Subject to free plan constraints.
Third-Party API Providers (The Dog API, The Cat API, Stripe)	External services providing pet data and mock payment capabilities.

3.2 Use Cases

3.2.1 Graphic Use Case Model

The PetPulse Web system supports four primary use cases across two main actor types: the general User (Pet Owner) and the Site Administrator. The use cases are:

- UC-01: Browse Product Information
- UC-02: Register for an Account
- UC-03: Login to Account
- UC-04: Complete Mock Subscription

Relationships: UC-03 extends UC-02 (login follows successful registration). UC-04 includes a redirect to Stripe test checkout. User Settings (profile update, password change, email change) are accessible only after UC-03 is completed.

3.2.2 Textual Description for Each Use Case

UC-01: Browse Product Information	
Use Case Name	Browse Product Information
Actors	Pet Owner
Description	User navigates to the Home page and browses sections explaining the PetPulse harness, its features, and benefits. May navigate to About or Features pages for more detail.
Data	Static page content (no user-provided data required).
Stimulus	User visits the site URL and scrolls through or navigates between pages.
Response	Each page loads and displays its content correctly.
Comments	All content is statically rendered; no external service dependency.

UC-02: Register for an Account	
Use Case Name	Register for an Account
Actors	Pet Owner
Description	User creates a new account by providing an email address and password, then verifies via email.
Data	Email address, password, OTP code.
Stimulus	User submits the signup form with valid credentials.
Response	Supabase creates a user record, sends an OTP email, and upon verification logs the user in with a JWT session.
Comments	If email is already registered, an error is displayed. If OTP is incorrect, re-entry is prompted.

UC-03: Login to Account	
Use Case Name	Login to Account
Actors	Pet Owner
Description	Registered user authenticates using email and password credentials.
Data	Email address, password.
Stimulus	User opens the login dialog and submits valid credentials.
Response	Supabase authenticates the user, establishes a JWT session, and the navbar updates to reflect the logged-in state.
Comments	Invalid credentials produce a generic error message. Protected routes become accessible after login.

UC-04: Complete Mock Subscription	
Use Case Name	Complete Mock Subscription
Actors	Pet Owner
Description	User selects a subscription plan and initiates a mock Stripe checkout process.
Data	Selected plan (Basic or Pro); no real payment data collected.
Stimulus	User clicks the Subscribe button on the /subscribe page.
Response	Stripe mock checkout handler triggers. No real payment is processed.
Comments	Stripe is in test mode only. If Stripe is unavailable, an error message is displayed.

3.3 Rationale for the Use Case Model

The use case model was designed to reflect the actual interactions of the two primary user groups with the PetPulse Web application. The four use cases cover the full range of public-facing functionality: information browsing (UC-01), account lifecycle management (UC-02, UC-03), and the stripe mock payment flow (UC-04).

Internal administrative functions (Supabase dashboard access, deployment) are deliberately excluded from the use case model, as they are performed via developer tools rather than the public-facing interface.

3.4 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Static pages (Home, About, Subscribe) shall load within 3 seconds on a standard broadband connection using Next.js SSR.
NFR-02	Responsiveness	All pages shall render correctly at desktop (1024px+), tablet (768–1023px), and mobile (<768px) viewport widths.
NFR-03	Browser Compatibility	The application shall support Chrome, Firefox, Safari.
NFR-04	Security	All authentication is managed by Supabase.
NFR-06	Accessibility	UI components shall include clear button labels, readable font sizes, and high-contrast color combinations.

4. Architecture

4.1 Architectural Styles Used

PetPulse Web employs a Three-Tier Architecture combined with a component-based frontend design pattern:

- Presentation Tier: React components (using shadcn/ui and Tailwind CSS) rendered via Next.js. Static pages use Server-Side Rendering (SSR); interactive components (authentication dialogs, settings form, checkout) use Client-Side Rendering (CSR).
- Logic Tier: Next.js server functions handle routing, SSR data fetching (e.g., Dog/Cat API calls on the Features page), and act as a bridge between the frontend and Supabase client.
- Data Tier: Supabase (PostgreSQL) manages two tables, auth.users (managed by Supabase internally) and public.profiles (user-created, linked by foreign key to auth.users).

4.2 Architectural Model (Components and Interactions)

Presentation Tier — React / Next.js

Component	Rendering	Description
Home Page (/)	SSR	Marketing landing page with hero section, image carousel, live monitor simulation widget.
About Page (/about)	SSR	Mission statement, team info, and PetPulse system overview.
Features Page (/features)	SSR + CSR	Feature cards (static) + Dog API and Cat API breed cards.
Subscribe Page (/subscribe)	SSR	Basic and Pro pricing cards with Stripe mock checkout handlers.
User Settings (/user/settings)	CSR	Protected route: profile form (name, password, email) backed by Supabase.
Navbar	CSR	Persistent across all pages; reflects auth state (login/logout, user email, mobile menu).
Auth Components (Login/Signup)	CSR	Dialog (desktop) and drawer (mobile) for authentication flows.

Logic Tier: Next.js Server + Supabase Client

- SSR Data Fetching: Server-side props functions call The Dog API and The Cat API on Features page render.
- Auth Flow: supabase.auth.* methods handle sign-up (OTP), login, logout, and JWT session management.

- Profile CRUD: supabase.from('profiles') queries handle reading and updating user profile data.

Data Tier — Supabase (PostgreSQL)

Two tables are used:

- auth.users: Managed by Supabase. Stores UID, email, password hash (bcrypt), OTP state, and timestamps.
- public.profiles: Application-managed. Stores first name, last name, updated_at, and created_at, with a foreign key linking to auth.users UID. RLS ensures users can only read/write their own row.

External Services

- The Dog API: Server-side GET requests during SSR on the Features page. JSON breed data returned and rendered as cards.
- The Cat API: Same pattern as The Dog API, providing cat breed data.
- Stripe (Test Mode): JavaScript client library; CTA buttons on Subscribe page trigger mock checkout sessions. No real payments processed.

4.3 Technology, Software, and Hardware Used

Technology	Version / Detail	Role
Next.js	App Router	Full-stack framework; SSR, routing, bundle delivery
React	18+	UI component library
Tailwind CSS	v3	Utility-first CSS for responsive styling
shadcn/ui	Latest	Pre-built accessible UI components
Supabase	Free Plan	Auth, PostgreSQL DB, REST API
The Dog API	Free Student Tier	Dog breed and health data
The Cat API	Free Student Tier	Cat breed data
Stripe	Test Mode	Mock subscription checkout
npm	Latest	Package management
Git / GitHub	Latest	Version control

4.4 Rationale for Architectural Style and Model

The three-tier architecture was selected because it cleanly separates user-facing presentation logic from data access concerns, which simplifies testing and maintenance. Next.js was chosen specifically because it supports both SSR (for fast initial load of marketing pages) and CSR (for interactive components requiring client-side state) within a single framework, eliminating the need for a separate backend server.

Supabase was mandated by project constraints and provides a mature, well-documented Auth system including OTP verification, JWT session handling. The free plan was sufficient for the academic scope of the project.

The Dog API and Cat API were selected for their simplicity and direct relevance to the educational pet health goals of the website. Stripe test mode was chosen to demonstrate a realistic e-commerce workflow without requiring real financial infrastructure.

5. Design

5.1 User Interface Design

The PetPulse Web interface is designed as a modern, dark-themed marketing website with light mode support, emphasizing clarity, readability, and visual engagement. The design language follows a component-based system using shadcn/ui primitives styled with Tailwind CSS.

5.1.1 Navigation Bar

A persistent top navigation bar is present on all pages. It provides links to Home, Features, About, and Subscribe. When a user is authenticated, the navbar displays the user's email and provides access to User Settings and Logout. On mobile viewports, navigation links collapse into a hamburger menu drawer. The active page link is visually highlighted.

5.1.2 Home Page

The landing page includes: a hero section with the PetPulse tagline, an image carousel showcasing the harness, a live monitor simulation widget displaying animated health statistics (step count, heart rate, etc.), and feature preview cards. The page is fully Server-Side Rendered.

5.1.3 About Page

Static SSR page presenting a mission statement section and a product overview section describing PetPulse's target users (pet owners) and the system's role in proactive pet health monitoring. Includes team member bios.

5.1.4 Features Page

The Features page has two main sections: feature cards highlighting the major capabilities of the PetPulse system (health monitoring, activity tracking, veterinary insights, mobile app integration), and a breed data section displaying dog and cat breed cards fetched from The Dog API and The Cat API. Breed cards show breed image, name, and health attributes. Loading skeleton states and API error fallbacks are included.

5.1.5 Subscribe Page

Presents Basic and Pro subscription plan cards in a responsive grid. Each card shows the plan name, price, included features list, and a 'Subscribe' CTA button. Buttons are wired to placeholder Stripe checkout handlers. The page clearly indicates that no real payment is processed.

5.1.6 User Settings Page

A protected page accessible only to authenticated users. Contains three update sections: Name update (first name, last name both optional, but both required together), Password change (minimum length validation, confirmation matching), and Email change request (format validation, triggers Supabase email verification flow). All sections display success and error feedback messages.

5.1.7 Responsive Design

Device Type	Viewport Width	Layout Behavior
Desktop	1024px and above	Full navigation bar, multi-column card grids, side-by-side layouts.
Tablet	768px – 1023px	Adjusted grid columns, maintained readability.
Mobile	Below 768px	Collapsed hamburger menu, single-column layouts, stacked cards.

5.2 Components Design (Static and Dynamic Models)

Static Component Structure

The Next.js App Router organizes the application into route-mapped page files and shared component directories:

- /app — Route-mapped pages (page.tsx files for each route)
- /components — Reusable UI components (Navbar, AuthDialog, BreedCard, Carousel, LiveMonitorWidget, SubscribeCard, SettingsForm)
- /lib — Supabase client initialization and helper utilities
- /public — Static assets (images, icons)

Dynamic Component Behavior

Component	Behavior
BreedCard	Renders breed image (with fallback), name, and attributes from API response; shows skeleton loader during fetch; shows error state on API failure.
LiveMonitorWidget	Uses setInterval to simulate changing health metrics; values update on a timer; cleared on component unmount.
Carousel	Arrow and dot navigation controls; auto-advances on timer; touch swipe supported on mobile.
Navbar	Subscribes to Supabase auth state changes; updates displayed user info reactively without page reload.

5.3 Database Design

The PetPulse Web database consists of two tables hosted on Supabase (PostgreSQL):

auth.users (Supabase-Managed)

Field	Type	Description
id	UUID (PK)	Auto-generated unique identifier for each user.
email	String	User's email address; used for authentication.
encrypted_password	String	bcrypt hash of the user's password; managed by Supabase.
confirmed_at	Timestamp	When OTP email was verified.
created_at	Timestamp	Account creation timestamp.
last_sign_in_at	Timestamp	Most recent login timestamp.

public.profiles (Application-Managed)

Field	Type	Description
id	UUID (PK, FK → auth.users.id)	Links profile row to the authenticated user's UID.
first_name	String (nullable)	User's first name; optional field.
last_name	String (nullable)	User's last name; optional field.
updated_at	Timestamp	Last time the profile row was modified.
created_at	Timestamp	When the profile row was first created.

External data (Dog API, Cat API breed data; Stripe checkout sessions) is not persisted in the application database. It is fetched at runtime and rendered client-side only.

5.4 Rationale for Detailed Design Models

The component-based design was chosen because it directly maps to React's programming model and promotes reusability. For example, the BreedCard component is used for both dog and cat data with no duplication. The database design is intentionally minimal. Supabase's auth.users table handles all authentication-critical data securely, while the application-owned public.profiles table only stores non-sensitive supplementary profile fields. This separation keeps the application's database responsibility small and leverages Supabase's hardened auth infrastructure.

5.5 Traceability from Requirements to Detailed Design Models

Requirement ID	Requirement	Design Component(s)
R1	Provide a landing page introducing PetPulse	Home Page, Carousel, LiveMonitorWidget, Hero Section
R2	Provide educational information about PetPulse	About Page, mission section, team bios
R3	Display PetPulse system capabilities	Features Page, Feature Cards (static React components)
R4	Display dog and cat breed/health information	Features Page, BreedCard (Dog API + Cat API sections)
R5	Allow users to create accounts	AuthDialog/Drawer (Sign-Up), Supabase auth.users table
R6	Allow users to log in securely	Supabase Auth + JWT
R7	Allow users to edit profile information	SettingsForm, public.profiles table
R8	Maintain user login session across pages	Supabase JWT, Navbar auth state subscription
R9	Display subscription pricing tiers	Subscribe Page, SubscribeCard components
R10	Allow users to initiate mock subscription checkout	Subscribe Page, CTA buttons, Stripe Test Mode handlers
R11	Ensure responsive layout across devices	Tailwind CSS responsive grid, Navbar mobile menu
R12	Provide error handling for external services	BreedCard error state, loading skeletons, fallback messages

6. Test Management

6.1 Complete List of System Test Cases

All test cases are organized by feature area. Each case includes an ID, test input, expected output, and description.

6.1.1 Pages and Navigation

ID	Test Input	Expected Output	Description
TC-01	Open /	Home page loads with hero, carousel, live monitor, and CTA buttons	Home page loads correctly
TC-02	Open /about	About page shows mission and team members	About page loads correctly
TC-03	Open /features	Features page shows feature panel and breed cards	Features page loads correctly
TC-04	Open /subscribe	Subscribe page shows Basic and Pro plans	Subscribe page loads correctly

6.1.2 Authentication

ID	Test Input	Expected Output	Description
TC-05	Click Login on desktop	Login dialog opens	Desktop login UI works
TC-06	Click Login on mobile	Login drawer opens	Mobile login UI works
TC-07	Enter invalid email in login form	Submit button disabled or login blocked	Login email validation works
TC-08	Enter valid login credentials	User logs in and navbar updates	Login works
TC-09	Enter wrong password	Error message is shown	Login error handling works
TC-10	Enter valid signup info	OTP step appears	Signup flow starts correctly
TC-11	Click Sign out	User is logged out and returned to home	Logout works

6.1.3 User Settings

ID	Test Input	Expected Output	Description
TC-12	Open settings page	Current profile info loads into the form	Settings data loads
TC-13	Update first and last name	Name is saved and success message appears	Name update works

ID	Test Input	Expected Output	Description
TC-14	Enter invalid new email	Email change request is blocked	Email validation works
TC-15	Enter valid new email	Verification request is sent	Email change request works

6.1.4 External API and Breed Cards

ID	Test Input	Expected Output	Description
TC-16	Open /features with valid API keys	Dog and cat breed cards load	Breed API data displays correctly
TC-17	Open /features with broken API	Error message shown instead of cards	API error handling works

6.1.5 UI Components and Theme

ID	Test Input	Expected Output	Description
TC-18	Toggle light/dark theme	Theme changes correctly	Theme toggle works
TC-19	Open app on mobile width	Mobile menu and responsive layout work	Responsive design works
TC-20	Click Subscribe button	Subscribe handler runs; no payment processed	Subscribe button behavior is correct

6.1.6 Error Handling and Build Checks

ID	Test Input	Expected Output	Description
TC-21	Cause Supabase error during settings update	Error message shown; app does not crash	Settings error handling works
TC-22	Try login/signup with bad credentials	Error message is shown	Auth error handling works

6.2 Traceability of Test Cases to Use Cases

Use Case	Test Cases
UC-01: Browse Product Information	TC-01, TC-02, TC-03, TC-04
UC-02: Register for an Account	TC-10
UC-03: Login to Account	TC-05, TC-06, TC-07, TC-08, TC-09, TC-11

Use Case	Test Cases
UC-04: View Educational Pet Health Content	TC-16, TC-17
UC-05: Complete Mock Subscription	TC-20
User Settings (Post-Login)	TC-12, TC-13, TC-14, TC-15, TC-21
UI / Cross-Cutting	TC-18, TC-19, TC-20, TC-22

6.3 Techniques Used for Test Case Generation

- Equivalence Partitioning: Inputs were divided into valid and invalid partitions (e.g., valid/invalid email formats, password length classes), reducing test case count while maintaining coverage.
- Boundary Value Analysis: Length-constrained fields were tested at their boundaries, for example, passwords at 7 characters (invalid) and 8 characters (valid).
- Use Case-Based Testing: Each use case was decomposed into primary flow and exception flow tests, covering both happy paths and key error paths.
- State-Based Testing: Authentication state transitions (logged-out → signed-up → OTP-verified → logged-in → logged-out) were tested as a sequence.

6.4 Test Results and Assessments

Feature Area	Test Cases	Passed	Failed	Notes
Pages & Navigation	TC-01 to TC-04	4	0	All pages load correctly.
Authentication	TC-05 to TC-11	7	0	Login, signup, and logout all functional.
User Settings	TC-12 to TC-15	4	0	Name update and email change validated correctly.
External API & Breed Cards	TC-16 to TC-17	2	0	API fetch and error state work correctly.
UI Components & Theme	TC-18 to TC-20	3	0	Theme toggle, responsive layout, and subscribe button all pass.
Error Handling	TC-21 to TC-22	2	0	Supabase and auth error handling confirmed.
Total	22	22	0	All test cases passed.

The test suite covers all core functional areas, with valid and invalid input paths exercised for all form components and authentication state transitions verified end-to-end. Performance and load testing were out of scope.

6.5 Defect Reports

Minor issues identified and resolved during the testing phase are summarized below:

Defect ID	Description	Severity	Resolution
DEF-001	Mobile navbar menu did not close after navigating to a new page.	Low	Added router event listener to close mobile menu on route change. Resolved.
DEF-002	Cat API breed images occasionally returned broken URLs, causing broken image icons.	Low	Added onError handler on breed card images to display a fallback placeholder. Resolved.

7. Conclusion

7.1 Outcomes of the Project

All primary goals defined in the Software Requirements Specification were achieved:

- A fully functional, responsive marketing and educational website for the PetPulse system was delivered, covering Home, About, Features, Subscribe, and User Settings pages.
- Secure user authentication was implemented via Supabase.
- Educational pet health content from The Dog API and The Cat API is displayed dynamically with appropriate loading and error states.
- A mock subscription workflow using Stripe test mode demonstrates the e-commerce capability of the platform without processing real payments.

7.2 Lessons Learned

- Next.js App Router learning curve: The distinction between server and client components required careful planning, misplacing client-only hooks (useState, useEffect) in server components caused early errors and refactoring.
- External API dependency requires early error handling investment: The Dog API and Cat API integration initially lacked error states. Adding robust loading/error UI retroactively took more effort than building it in from the start.

7.3 Future Development

Given additional time and resources, the following enhancements would be prioritized:

- Full Stripe checkout integration: Replace placeholder handlers with a real Stripe Checkout Session flow, including a post-payment confirmation page with webhook handling.
- Expanded pet health content: Give The Cat API a more prominent role in the educational content section alongside The Dog API.
- Mobile application integration: Link the website to the PetPulse mobile app, allowing users who sign up on the web to seamlessly transition to the mobile experience.

References

[1] Next.js Documentation. App Router. https://nextjs.org/docs

[2] Supabase Documentation. Authentication and Database. https://supabase.com/docs

[3] The Dog API. https://docs.thedogapi.com

[4] The Cat API. https://docs.thecatapi.com

[5] Stripe. Test Mode Documentation. https://stripe.com/docs/testing

[6] shadcn/ui. Component Library. https://ui.shadcn.com

[7] Tailwind CSS Documentation. https://tailwindcss.com/docs